

Table of Contents

Scope	2
Approach	2
Purpose	2
Development.....	2
Basic Design.....	3
Levels	4
Level Implementation	5
How Rooms Are Stored In The Game (C#/Delphi/Visual Basic)	6
How Rooms Are Stored In The Game (COBOL).....	7
The Parser.....	8

Rog's Text Adventure - Design

Created By: **Roger Williams**

Scope

To create an old school text based adventure game as generic in design as possible so it can be translated/converted into other languages.

In three main phases:

- 1 – basic game with movement commands only
- 2 – more involved mechanics with objects in the room and ability to interact with them better parser
- 3 – advanced with entities and room events, more intense parser

Approach

- To create the game as *simply* as possible. Avoiding where possible language specific commands and objects.
- To use the simplest method for storing levels etc.
- To use the console to provide a simple generic universal output system
- To use ASCII text only no bitmaps or pixel drawing, only colour changes allowed where possible

Purpose

To have the game created in Visual Basic Dot Net, then converted into:

- C#
- Delphi
- COBOL
- *Javascript

Development

- The Visual Basic/C#/Javascript versions will be written in Visual Studio 2022
- The Delphi version will be written in Delphi 12
- The COBOL version will be written using OpenCOBOLIDE and compiled with GnuCOBOL 3.2.0

Hence the design needs to be generic enough to be reachable by all the chosen languages.

*The Javascript needs its own design document because of its inherent limitations particularly because Javascript cannot read files directly and has no interactive console, so decision needs to be made as to making a Windows Forms version or a HTML version.

Basic Design

Intro:

The intro is comprises of two text files:

- Introscr1.txt
- Introscr2.txt

For C#/Delphi/Visual Basic

Each file is 120 columns wide and 40 rows long, as the console can be resized in Windows this allows for a much bigger display which suits modern displays.

The screens will be loaded into memory and displayed in turn in alternating colours, creating a visually pleasing effect, Windows API SLEEP will be used in the intro and anywhere in the game where something needs to be displayed to the user *but* user interaction is not required, so the user can see the message.

For COBOL

As COBOL is a transactional language it does not have any native facility to “handle” the console window, there is no API for setting the size as COBOL isn’t intended for that, so the design of the game files needs to conform to COBOL’s default display limitations of 80 columns wide and 30 columns long.

Plus in COBOL there is no facility to clear the console screen, set its size or set colours.

The screens will be loaded into memory and displayed in turn in, a delay loop will be used in the intro and anywhere in the game where something needs to be displayed to the user *but* user interaction is not required, so the user can see the message.

Levels

Levels follow this format:

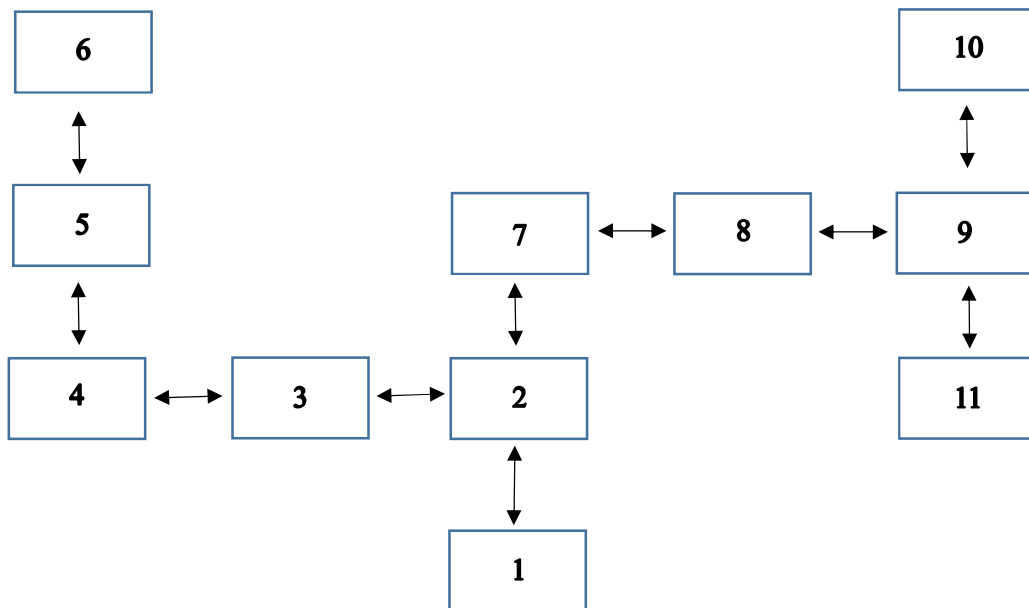
- Room ID
- Room ID to the North
- Room ID to the South
- Room ID to the East
- Room ID to the West
- Room description lines

Room ID fields numeric, room description lines are strings.

This format means complex levels can be stored with ease, as level layouts on paper best represent a binary tree, so storing in a flat file poses its own issues. Which is why each room has a unique ID and each available direction in the room has a room ID adjacent to it, using zero to represent no room adjacent.

Meaning levels can be stored in *any* room order and the rooms will still be displayed to the player in the correct order.

Example level design from paper with room IDs:



Levels are not bound to any fixed compass direction. Room one could claim to have a door to the North, or even the East.

Level Implementation

For C#/Delphi/Visual Basic

Levels are stored in text files and stored as records in this format:

- Room ID
- Room ID to the North
- Room ID to the South
- Room ID to the East
- Room ID to the West
- Room description lines

The room description lines are 20 lines of text with a maximum of 120 characters wide.

For COBOL

Because COBOL does not like the file format above, instead each room is written as *one* line in the text file, with the numeric values having a fixed length of 2 and the room description lines a fixed length of 80.

This means if numeric values are less than 10 then zero needs to be prefixed, and all strings needs to be padded to 80 characters in length. E.g. (obviously without the word wrap!):

```
0100030002It is a cold Monday morning. The rain beats heavily against the window, turning everything grey and miserable.
You stand by your desk, Windows 11 is chuntering to itself (as it does) and you know it will be at least 15 minutes before its ready to be used.
Now you know why SSDs were invented, if only you had one installed!                               The cheap metal table wobbles
slightly under the rigour in which Windows 11 is mercilessly hammering the harddrive simply because you switched on the computer.
Rumours of a new version of Windows coming, soon usual promises, higher speed machine required to use the same version of Office as the last one...
The bland magnolia walls of the office and the solitary strip light give an almost film noir feeling to the room.
Another huge pile of Windows issues awaits you today, more joys with printers Windows forgets exist, domain networks suddenly becoming Public for no
reason, shared network drives suddenly having incorrect credentials despite them being correct...
There is a door to your west, and another to the south
```

Also unlike the Delphi/VB/C# versions where levels can be easily created in a text editor, because of the special formatting COBOL requires, a special level creator for internal use will need to be created.

Note: COBOL has 25 lines of 80 fixed length characters which is narrower than the C#/Delphi/Visual Basic versions due to its console limitations but also longer.

How Rooms Are Stored In The Game (C#/Delphi/Visual Basic)

In the game the level rooms will be (where possible) stored using a record or a class, the properties of which will be the same e.g. for Visual Basic:

```
Public Class clsGameRooms
  Public ID As Integer = 0
  'next 4 properties determine which room this one leads to 0=no room!
  Public NextRoomNorth As Integer = 0
  Public NextRoomSouth As Integer = 0
  Public NextRoomEast As Integer = 0
  Public NextRoomWest As Integer = 0
  'used for text to describe room to player
  Public Desc1 As String
  Public Desc2 As String = ""
  Public Desc3 As String = ""
  Public Desc4 As String = ""
  Public Desc5 As String = ""
  Public Desc6 As String = ""
  Public Desc7 As String = ""
  Public Desc8 As String = ""
  Public Desc9 As String = ""
  Public Desc10 As String = ""
  Public Desc11 As String = ""
  Public Desc12 As String = ""
  Public Desc13 As String = ""
  Public Desc14 As String = ""
  Public Desc15 As String = ""
  Public Desc16 As String = ""
  Public Desc17 As String = ""
  Public Desc18 As String = ""
  Public Desc19 As String = ""
  Public Desc20 As String = ""
End Class
```

This class in the game would be used to store *each* room in its own class, this data (class for each room) would be stored in a list:

```
Dim lstRooms As New List(Of clsGameRooms)    'collection of all rooms in the level
```

This concept is to be used and adapted in Delphi as C# (as part of the Dot Net Framework) already supports Lists.

How Rooms Are Stored In The Game (COBOL)

As COBOL does not have Lists use its inherent tables structure instead:

```
01 REC-ROOM-INTERNAL.
  03 REC-ROOM OCCURS 40 TIMES.
    05 INT-ROOMID PIC 99 VALUE ZEROES.
  * 'next 4 propoerties determine which room this one leads to 0=no room!
    05 INT-NEXTROOMNORTH PIC 99 VALUE ZEROES.
    05 INT-NEXTROOMSOUTH PIC 99 VALUE ZEROES.
    05 INT-NEXTROOMEAST PIC 99 VALUE ZEROES.
    05 INT-NEXTROOMWEST PIC 99 VALUE ZEROES.
  *used for text to describe room to player
    05 STR-DESC-INT1 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT2 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT3 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT4 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT5 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT6 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT7 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT8 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT9 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT10 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT11 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT12 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT13 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT14 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT15 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT16 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT17 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT18 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT19 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT20 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT21 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT22 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT23 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT24 PIC X(80) VALUE SPACES.
    05 STR-DESC-INT25 PIC X(80) VALUE SPACES.
```

Note: REC-ROOM OCCURS 40 TIMES creates REC-ROOM as an effective array of 40 room records.

The Parser

This should split the user data entry into strings where possible, then analyse these strings to see if they conform to some simple grammar rules.

For Phase one these will be:

- every phrase should contain a verb
- every verb should either have an adjective e.g. open door or a preposition e.g. while or a noun e.g. key, after it

Testing will need to be done on whether a sentence can be accurately interpreted, but for phase one it only needs to recognise movement commands e.g. move south, go north.

More advanced grammar checks would be implemented in the later versions of the game.

For C#/Delphi/Visual Basic

The parser will be its own class, not because the game needs to create multiple parsers, but because it makes the code easier to debug and read if the parser is in its own code file,

For COBOL

As the GNUCOBOL does not support classes the COBOL program will have the parser class built into it as standard code, with standard paragraphs for each class method and variables for the class properties.